INNOMATICS®
RESEARCH LABS

INNOVATION. AUTOMATION. ANALYTICS

PROJECT ON

Grocery Store Management System

About me

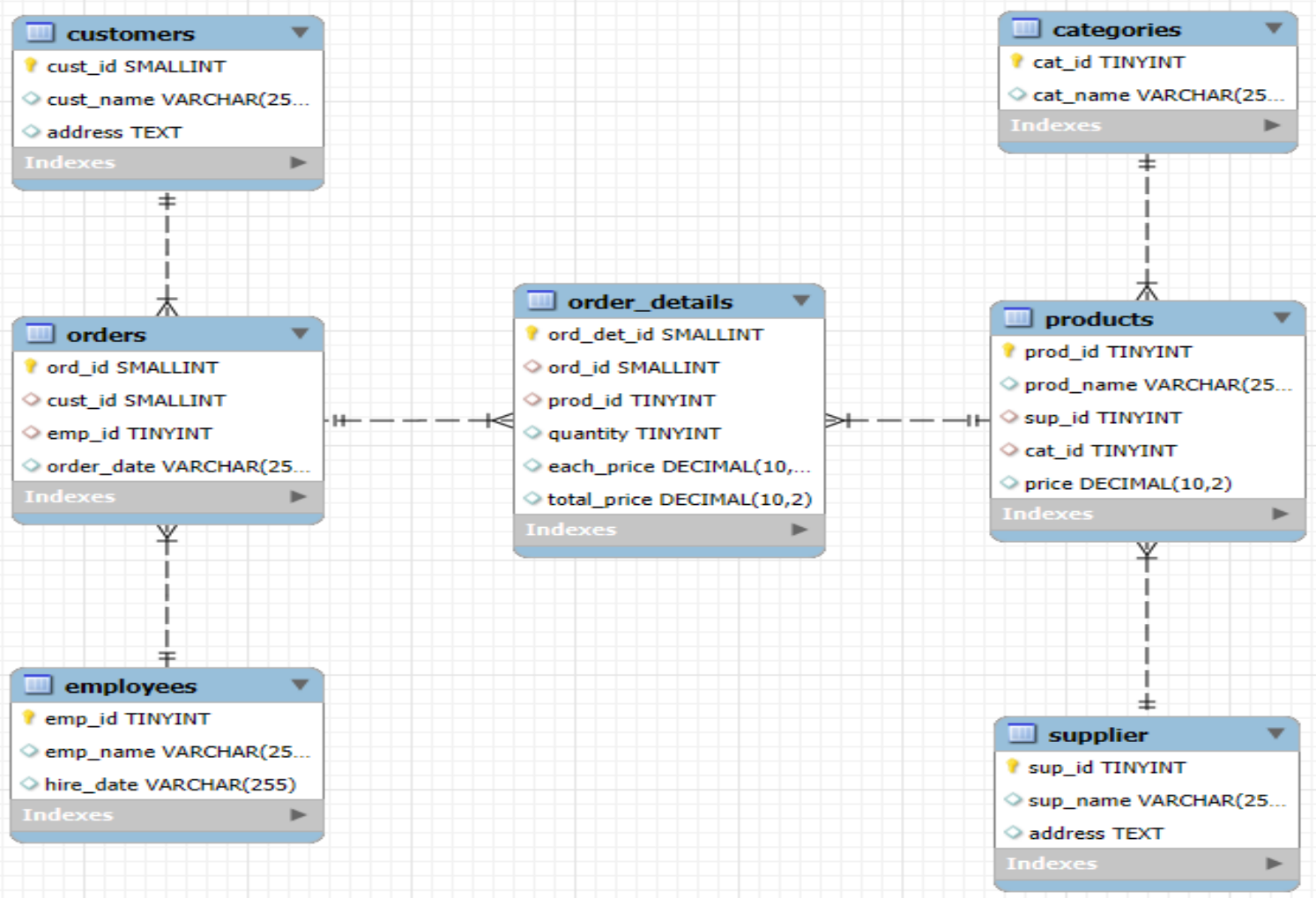
- I have Pursed my Master’s in Data Science at Essex University, Colchester, UK.
- I have successfully completed 3 months of Internship from Myupdate24.
- As a Data Analyst with hands-on experience in Internship, I have implemented technique like data cleaning, analysis, and visualization. Strong proficiency in SQL, Python, Excel, and Power BI to extract insights, build dashboards, and support data driven decision-making. Adept at translating complex data into clear business insights.
- Also looking forward to learn more in the field of Data Science and Agentic AI.
- Looking of opportunities where I can implement my skills.

Objective

This SQL project aims to design and implement a relational database for grocery store management, enabling comprehensive data analysis across customers, products, orders, and suppliers. The objective is to master SQL techniques including joins, aggregations, and subqueries while extracting meaningful business insights such as top customers, best-selling products, and revenue trends. The project demonstrates how data-driven decision-making can optimize inventory management, improve customer retention strategies, and enhance overall operational efficiency in retail grocery businesses.

ER Diagram and schema explanation

The Grocery Store Management database consists of **7 interconnected tables** designed to track grocery store operations: **SUPPLIER, CATEGORIES, EMPLOYEES, CUSTOMERS, PRODUCTS, ORDERS, and ORDER_DETAILS**. **PRODUCTS** acts as a bridge connecting suppliers and categories to inventory, while **ORDERS** links customers and employees to transactions. **ORDER_DETAILS** contains line items, capturing which products were ordered in what quantities at what prices. All tables are connected through **Foreign Key relationships with CASCADE constraints** to maintain data integrity and prevent orphaned records. This normalized **3NF schema** enables comprehensive analysis of customer behavior, product performance, supplier effectiveness, and employee productivity across the entire retail operation.



Tables

Category Table

Field	Type	Null	Key	Default	Extra
cat_id	tinyint	NO	PRI	NULL	auto_increment
cat_name	varchar(255)	YES		NULL	

Customer Table

Field	Type	Null	Key	Default	Extra
cust_id	smallint	NO	PRI	NULL	auto_increment
cust_name	varchar(255)	YES		NULL	
address	text	YES		NULL	

Orders Table

Field	Type	Null	Key	Default	Extra
ord_id	smallint	NO	PRI	NULL	auto_increment
cust_id	smallint	YES	MUL	NULL	
emp_id	tinyint	YES	MUL	NULL	
order_date	varchar(255)	YES		NULL	

Order Details Table

Field	Type	Null	Key	Default	Extra
ord_det_id	smallint	NO	PRI	NULL	auto_increment
ord_id	smallint	YES	MUL	NULL	
prod_id	tinyint	YES	MUL	NULL	
quantity	tinyint	YES		NULL	
each_price	decimal(10,2)	YES		NULL	
total_price	decimal(10,2)	YES		NULL	

Products Table

Field	Type	Null	Key	Default	Extra
prod_id	tinyint	NO	PRI	NULL	auto_increment
prod_name	varchar(255)	YES		NULL	
sup_id	tinyint	YES	MUL	NULL	
cat_id	tinyint	YES	MUL	NULL	
price	decimal(10,2)	YES		NULL	

Employee Table

Field	Type	Null	Key	Default	Extra
emp_id	tinyint	NO	PRI	NULL	auto_increment
emp_name	varchar(255)	YES		NULL	
hire_date	varchar(255)	YES		NULL	

Supplier Table

Field	Type	Null	Key	Default	Extra
sup_id	tinyint	NO	PRI	NULL	auto_increment
sup_name	varchar(255)	YES		NULL	
address	text	YES		NULL	

customer insights

1.1. How many unique customers have placed orders ?

```
SELECT COUNT(DISTINCT(cust_id)) AS "Unique Customer"
FROM orders;
```

Output :-

	Unique Customer
▶	156

1.2. Which customers have placed the highest number of orders ?

```
SELECT c.cust_id,
       c.cust_name,
       COUNT(o.ord_id) AS total_orders
FROM customers c
JOIN orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id, c.cust_name
ORDER BY total_orders DESC limit 1;
```

Output :-

	cust_id	cust_name	total_orders
▶	165	Jyotika	7

1.3. What is the total and average purchase value per customer?

```sql
SELECT c.cust_id,
       c.cust_name,
       SUM(od.total_price) AS total_spent,
       AVG(od.total_price) AS avg_spent
FROM customers c
JOIN orders o ON c.cust_id = o.cust_id
JOIN order_details od ON o.ord_id = od.ord_id
GROUP BY c.cust_id, c.cust_name;
```

Output :-

cust_id	cust_name	total_spent	avg_spent
1	Aditi Shetty	1577.86	1577.860000
2	Isha Reddy	1299.62	649.810000
3	Chetan Rao	7693.41	854.823333
5	Isha Rao	3327.05	1663.525000
7	Eshwar Iyer	9188.45	656.317857
8	Deepa Reddy	7929.13	881.014444
13	Bala Krishnan	1161.66	580.830000
14	Jaya Shetty	1932.94	966.470000
15	Aditi Gowda	948.15	474.075000
16	Gita Naidu	173.08	173.080000
17	Kiran Pillai	1398.62	699.310000
18	Jaya Rao	3485.80	871.450000
19	Chetan Naidu	11256.82	1125.682000
21	Bala Menon	380.72	190.360000

1.4. Who are the top 5 customers by total purchase amount?

```sql
SELECT c.cust_id,
       c.cust_name,
       SUM(od.total_price) AS total_revenue
FROM customers c
JOIN orders o ON c.cust_id = o.cust_id
JOIN order_details od ON o.ord_id = od.ord_id
GROUP BY c.cust_id, c.cust_name
ORDER BY total_revenue DESC
LIMIT 5;
```

Output :-

cust_id	cust_name	total_revenue
19	Chetan Naidu	11256.82
166	Kapila	11099.51
67	Eshwar Rao	10819.96
61	Aditi Rao	10230.64
7	Eshwar Iyer	9188.45

Product Performance

2.1. How many products exist in each category ?

```
SELECT cat.cat_name,
       COUNT(p.prod_id) AS total_products
FROM categories cat
JOIN products p ON cat.cat_id = p.cat_id
GROUP BY cat.cat_name
order by total_products desc;
```

Output :-

cat_name	total_products
Grains & Cereals	18
Snacks & Confectioneries	17
Dairy Products	6
Personal Care	6
Household	3

2.2. What is the average price of products by category?

```sql
SELECT cat.cat_name,
       AVG(p.price) AS avg_price
FROM categories cat
JOIN products p ON cat.cat_id = p.cat_id
GROUP BY cat.cat_name
order by avg_price desc ;
```

Output :-

	cat_name	avg_price
▶	Dairy Products	366.943333
	Personal Care	364.991667
	Household	363.336667
	Grains & Cereals	287.673333
	Snacks & Confectioneries	278.892353

2.3.Which products have the highest total sales volume (by quantity)?

```
SELECT p.prod_name,
       SUM(od.quantity) AS total_quantity_sold
FROM products p
JOIN order_details od ON p.prod_id = od.prod_id
GROUP BY p.prod_name
ORDER BY total_quantity_sold DESC limit 1;
```

Output :-

	prod_name	total_quantity_sold
▶	Bath Soap	60

2.4. What is the total revenue generated by each product?

```
SELECT p.prod_name,
       SUM(od.total_price) AS total_revenue
FROM products p
JOIN order_details od ON p.prod_id = od.prod_id
GROUP BY p.prod_name
ORDER BY total_revenue DESC;
```

Output :-

prod_name	total_revenue
Hand Sanitizer	27787.76
Biscuits	20995.92
Moong Dal	19695.02
Toothpaste	19688.95
Mustard Seeds	19516.68
Cashews	18561.92
Butter	18548.40
Cheese Slices	18519.61
Turmeric Powder	17784.29
Soya Sauce	16985.38
Toilet Cleaner	16776.90
White Bread	16576.21
Paneer	15980.84
Detergent Powder	14848.72
Brown Bread	14538.04

2.5. How do product sales vary by category and supplier?

```sql
SELECT cat.cat_name,
       s.sup_name,
       SUM(od.total_price) AS total_sales
FROM order_details od
JOIN products p ON od.prod_id = p.prod_id
JOIN categories cat ON p.cat_id = cat.cat_id
JOIN supplier s ON p.sup_id = s.sup_id
GROUP BY cat.cat_name, s.sup_name
ORDER BY total_sales DESC;
```

Output :-

cat_name	sup_name	total_sales
Personal Care	Aarya	69378.41
Grains & Cereals	Aarya	67701.10
Snacks & Confectioneries	Aarya	65538.71
Snacks & Confectioneries	Suresh	65307.14
Dairy Products	Sai	50740.60
Grains & Cereals	Karthik	39473.49
Snacks & Confectioneries	Aarav Sharma	26948.15
Grains & Cereals	Suresh	26248.89
Household	Karthik	22767.59
Dairy Products	Aarya	18519.61
Grains & Cereals	Sai	18018.02
Snacks & Confectioneries	Sai	17103.15
Household	Sai	16776.90
Dairy Products	Karthik	11100.45
Personal Care	Sai	10949.84

3. Sales and Order Trends

3.1 How many orders have been placed in total?

```
SELECT COUNT(*) AS total_orders
FROM orders;
```

Output :-

	total_orders
▶	300

3.2 What is the average value per order?

```sql
SELECT AVG(order_total) AS avg_order_value
FROM (
    SELECT ord_id,
           SUM(total_price) AS order_total
    FROM order_details
    GROUP BY ord_id
) t;
```

Output :-

	avg_order_value
▶	2153.632539

3.3 On which dates were the most orders placed?

```
SELECT order_date,
       COUNT(ord_id) AS total_orders
FROM orders
GROUP BY order_date
ORDER BY total_orders DESC limit 1;
```

Output :-

	order_date	total_orders
▶	9/10/2022	4

3.4 What are the monthly trends in order volume and revenue?

```sql
SELECT
month(STR_TO_DATE(order_date, '%m/%d/%Y')) AS ord_month,
COUNT(DISTINCT o.ord_id) AS ord_volume,
SUM(od.total_price) AS total_revenue
FROM orders o
INNER JOIN order_details od
ON o.ord_id= od.ord_id
GROUP BY ord_month
ORDER BY ord_month;
```

Output :-

ord_month	ord_volume	total_revenue
1	30	70312.45
2	28	66929.42
3	27	45977.16
4	11	29118.54
5	19	41305.62
6	14	27378.69
7	21	48674.66
8	20	36045.01
9	23	52626.61
10	12	25917.32
11	19	46141.33
12	32	60903.12

3.5 How do order patterns vary across weekdays and weekends?

```
SELECT
     weekday((STR_TO_DATE(order_date, '%m/%d/%Y')))AS days,
    dayname((STR_TO_DATE(order_date, '%m/%d/%Y')))AS weekdays,
    COUNT(ord_id) AS total_orders
FROM orders
GROUP BY days,weekdays
order by days,weekdays asc;
```

Output :-

days	weekdays	total_orders
0	Monday	37
1	Tuesday	31
2	Wednesday	51
3	Thursday	39
4	Friday	52
5	Saturday	42
6	Sunday	48

4. Supplier Contribution

4.1 How many suppliers are there in the database?

```
SELECT COUNT(*) AS total_suppliers
FROM supplier;
```

Output :-

	total_suppliers
▶	5

4.2 Which supplier provides the most products?

```
SELECT s.sup_name,
       COUNT(p.prod_id) AS total_products
FROM supplier s
JOIN products p ON s.sup_id = p.sup_id
GROUP BY s.sup_name
ORDER BY total_products DESC limit 1;
```

Output :-

	sup_name	total_products
▶	Aarya	18

4.3 What is the average price of products from each supplier?

```
SELECT s.sup_name,
       AVG(p.price) AS avg_price
FROM supplier s
JOIN products p ON s.sup_id = p.sup_id
GROUP BY s.sup_name;
```

Output :-

sup_name	avg_price
Aarav Sharma	271.366667
Sai	342.672000
Aarya	319.326667
Suresh	281.818000
Karthik	288.225556

4.4 Which suppliers contribute the most to total product sales (by revenue)?

```sql
SELECT s.sup_name,
       SUM(od.total_price) AS total_revenue
FROM supplier s
JOIN products p ON s.sup_id = p.sup_id
JOIN order_details od ON p.prod_id = od.prod_id
GROUP BY s.sup_name
ORDER BY total_revenue DESC limit 1;
```

Output :-

	sup_name	total_revenue
▶	Aarya	221137.83

5. Employee Performance

5.1 How many employees have processed orders?

```
SELECT COUNT(DISTINCT emp_id) As NO_of_employee_pro
FROM orders;
```

Output :-

	No_of_employee_pro
▶	10

5.2 Which employees have handled the most orders?

```sql
SELECT e.emp_name,
       COUNT(o.ord_id) AS total_orders
FROM employees e
JOIN orders o ON e.emp_id = o.emp_id
GROUP BY e.emp_name
ORDER BY total_orders DESC limit 1;
```

Output :-

	emp_name	total_orders
▶	Diya Sharma 1	38

5.3 What is the total sales value processed by each employee?

```sql
SELECT e.emp_name,
       SUM(od.quantity) as totalquantity,
       SUM(od.total_price) AS total_revenue
FROM employees e
JOIN orders o ON e.emp_id = o.emp_id
JOIN order_details od ON o.ord_id = od.ord_id
GROUP BY e.emp_name
ORDER BY total_revenue DESC;
```

Output :-

emp_name	totalquantity	total_revenue
Aditya Singh 1	244	79252.29
Zara Verma 1	226	71562.76
Diya Sharma 1	218	67241.85
Pari Kumar 1	219	66818.39
Arjun Kumar 1	179	54018.31
Aarav Kumar 1	154	52602.88
Vihaan Singh 1	151	48577.88
Pari Sharma 1	145	40334.22
Arjun Verma 1	113	36716.84
Aditya Verma 1	123	34204.51

5.4 What is the average order value handled per employee?

```sql
SELECT e.emp_name,
       AVG(order_total) AS avg_order_value
FROM employees e
JOIN orders o ON e.emp_id = o.emp_id
JOIN (
    SELECT ord_id,
           SUM(total_price) AS order_total
    FROM order_details
    GROUP BY ord_id
) t ON o.ord_id = t.ord_id
GROUP BY e.emp_name
order by avg_order_value desc ;
```

Output :-

emp_name	avg_order_value
Aarav Kumar 1	2768.572632
Zara Verma 1	2650.472593
Aditya Singh 1	2330.949706
Pari Kumar 1	2227.279667
Vihaan Singh 1	2112.081739
Arjun Kumar 1	2077.627308
Diya Sharma 1	2037.631818
Arjun Verma 1	1835.842000
Pari Sharma 1	1833.373636
Aditya Verma 1	1554.750455

6. Order Details Deep Dive

6.1 What is the relationship between quantity ordered and total price?

```
SELECT quantity,
       AVG(total_price) AS avg_total_price
FROM order_details
GROUP BY quantity
ORDER BY quantity;
```

Output :-

quantity	avg_total_price
1	319.274516
2	595.716667
3	898.266377
4	1338.607143
5	1530.401121

6.2 What is the average quantity ordered per product?

```sql
SELECT p.prod_name,
       AVG(od.quantity) AS avg_quantity
FROM products p
JOIN order_details od ON p.prod_id = od.prod_id
GROUP BY p.prod_name
order by avg_quantity desc;
```

Output :-

prod_name	avg_quantity
Butter	4.5556
Toothpaste	3.6667
Potato Chips	3.6000
Tomato Ketchup	3.5000
Mustard Seeds	3.4615
Moong Dal	3.4000
Ghee	3.3750
Mango Pickle	3.3636
Bath Soap	3.3333
Chili Sauce	3.2500
Salt	3.2500
Facial Tissue	3.2143
Basmati Rice	3.2000
Detergent Pow...	3.2000
Mouth wash	3.2000
Sugar	3.2000
Green Tea	3.1818
Chocolate Bar	3.1818
Instant Noodles	3.1818
Biscuits	3.1765
Chapati	3.1250
Jaggery	3.0833
Red Chili Powder	3.0769
Cumin Seeds	3.0667
Almonds	3.0000

6.3 How does the unit price vary across products and orders?

```sql
SELECT p.prod_id,p.prod_name,od.each_price AS unit_price,COUNT(od.ord_id) AS times_ordered
FROM products p
INNER JOIN order_details od
    ON p.prod_id = od.prod_id
GROUP BY p.prod_id,od.each_price
ORDER BY p.prod_id,od.each_price;
```

Output :-

prod_id	prod_name	unit_price	times_ordered
1	Basmati Rice	358.98	10
2	Wheat Flour	255.50	15
3	Moong Dal	386.18	15
4	Chickpeas	353.50	7
5	Soybean Oil	172.81	11
6	Ghee	487.46	8
7	Paneer	484.27	11
8	Yogurt	111.61	7
9	Mango Pickle	182.50	11
10	Mixed Vegetable Pickle	133.51	7
11	Almonds	315.57	9
12	Cashews	441.95	16
13	Green Tea	166.26	11
14	Masala Tea	380.85	10
15	Coffee Powder	179.55	14
16	Sugar	409.62	5
17	Jaggery	200.85	12
18	Salt	210.36	8

Project Conclusion

- The **Grocery Store Management SQL Project** demonstrates the design and implementation of a **comprehensive relational database** with **7 interconnected tables**.
- The project enables **efficient data management and business intelligence**.
- Advanced SQL techniques such as **joins, aggregations, and subqueries** were used to analyze the data.
- The analysis helped extract **meaningful insights** including:
    - Customer purchasing behavior
    - Product performance metrics
    - Supplier contributions
    - Employee sales performance
- The project demonstrates proper **database normalization**, **referential integrity constraints**, and **CASCADE rules** to ensure data consistency.
- These structures support **complex analytical queries**.
- The findings provide **actionable insights** for:
    - Inventory optimization
    - Customer retention strategies
    - Operational efficiency improvements
- The project combines **technical database skills with practical business applications**, preparing professionals for **real-world data-driven decision-making in retail and e-commerce**.
- The complete deliverables include:
    - SQL scripts
    - ERD diagrams
    - Professional presentations
- The project demonstrates a **complete end-to-end analytical workflow**, from **database design to business insights generation**.

Challenges

- Understanding complex table relationships and applying the correct **INNER, LEFT, or RIGHT joins** to combine data from multiple tables without creating duplicates or losing information can be challenging for beginners.
- Managing **data integrity using foreign key constraints and CASCADE rules** requires careful planning, because accidental deletions or updates can propagate across related tables unexpectedly.
- Aggregating data across joined tables using **GROUP BY and HAVING clauses** may lead to incorrect results if **cardinality relationships** are not properly understood, often requiring multiple iterations to produce accurate metrics.
- Handling **date data stored as VARCHAR instead of DATE format** complicates time-based analysis and trend extraction, which requires additional **data type conversions** during queries.
- **Optimizing query performance** becomes important when working with larger datasets (such as **600+ order details**), requiring the use of **indexes** and avoiding inefficient nested subqueries.
- Balancing **database normalization and query complexity** is another challenge, because while **3NF eliminates redundancy**, it requires writing more complex **multi-table joins**, which demands stronger SQL skills and careful query planning.

Q&A

THANK YOU